

SOLIDWORKS Document Manager API (.NET) — Examples (1/4): Code_Example_CSharp

SOLIDWORKS API Documentation (converted from CHM)

DimXpert Example (C#)

This example shows how to call interfaces, methods, and properties in the SOLIDWORKS Document Manager API for parts with DimXpert features and annotations.

To learn how to build parts containing DimXpert features and annotations that work with a particular API, open the examples in the **Examples** section in the SOLIDWORKS Document Manager API topic of interest.

```
//-----
// Preconditions:
// 1. Read the SOLIDWORKS Document Manager API Getting Started topic
//    and ensure that the required DLLs have been registered.
// 2. Copy and paste this class into a C# console application in
//    Microsoft Visual Studio. Save the project with name, DimXpertExample_CSharp.
// 3. Create a part that contains DimXpert features and annotations (see NOTES below).
// 4. Add the SolidWorks.Interop.swdocumentmgr.dll reference to the project:
//    a. Right-click the project in Project Explorer.
//    b. Click Add Reference.
//    c. Click Browse.
//    d. Click:
//        install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll
// 5. Substitute your_part_with_dimensions with the path to the part you just created.
// 6. Substitute your_license_code with your license code.
// 7. Compile and run this program.
//
// Postconditions: Inspect the Immediate Window.
//
// NOTES:
// You must convert older parts to the latest supported
// version of SOLIDWORKS in order to get the DimXpert CAD identifiers
// for features and annotations using the API. To convert an older
// part to the latest supported version:
//
// 1. Open the part in a version of SOLIDWORKS that contains this API.
// 2. Save the part, clicking OK in the pop-up dialog to convert the part.
//    NOTE: If you are using a tutorial part, be sure to save it to another name.
// 3. Close the part.
//-----

using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Globalization;
using System.IO;
using System.Linq;
using System.Reflection;
using System.Runtime.CompilerServices;
using System.Security;
using System.Text;
using System.Threading.Tasks;
using Microsoft.VisualBasic;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swdocumentmgr;
namespace DimXpertExample_CSharp {
    partial class SolidWorksMacro
    {
        public void Main()
        {
            SwDMClassFactory classfac;
            SwDMAApplication docMgr;
            SwDMDocument5 swDoc;
            SwDmDocumentOpenError e;

            classfac = new SwDMClassFactory();

            // Substitute your license code here:
            docMgr = classfac.GetApplication("your_license_code");

            Debug.Print("Latest supported file version: " + docMgr.GetLatestSupportedFileVersion());
            string fileName;

            // Substitute the path to your DimXpert part here:
            fileName = "your_parts_full_pathname";

            swDoc = (SwDMDocument5) docMgr.GetDocument(fileName, SwDmDocumentType.swDmDocumentPart, true, out e);
            if ((swDoc == null))
            {
                Debug.Print(fileName + " not found");
                return;
            }
            Debug.Print("Loaded document version: " + swDoc.GetVersion());
            Debug.Print("Loaded document name: " + fileName);
            Debug.Print(" ");
            object[] vConfigNames;
            string configName;
            SwDMConfiguration12 configObj;
            int index;
            SwMDimXpertPart myDimXpertPart;
            object[] vFeatures;
            SwMDimXpertFeature myFeature;
        }
    }
}
```

```

object[] vAnnotations;
SwDMDimXpertAnnotation myAnnotation;
SwDMDimXpertBlockTolerances myBlockTolerance;
object[] vObject;
int featureIndex;
int annoIndex;
int vObjectIndex;
SwDMDimXpertFeature tempFeature;
SwDMDimXpertFeature tempFeature1;
SwDMDimXpertFeature endFeature1;
SwDMDimXpertFeature endFeature2;
SwDMDimXpertPlaneFeature planeFeature1;
SwDMDimXpertPlaneFeature planeFeature2;
swDmDimXpertDatum datum;
swDmDimXpertDistanceFosUsage_e fosUsage = 0;
swDmDimXpertMaterialConditionModifier_e modifier = 0;
swDmDimXpertBlockToleranceType_e bltType = 0;
double X = 0.0;
double Y = 0.0;
double Z = 0.0;
double i = 0.0;
double j = 0.0;
double k = 0.0;
double longi = 0.0;
double longj = 0.0;
double longk = 0.0;
double width = 0.0;
double length = 0.0;
double angle = 0.0;
double Distance = 0.0;
double Tolerance = 0.0;
double Value = 0.0;
bool isMax = false;
bool Enabled = false;
swDmDimXpertFeatureType_e feattype;
swDmDimXpertAnnotationType_e annotype;
SwDMDimXpertGeometricTolerance myGeoTol;
SwDMDimXpertDimensionTolerance myDimTol;
SwDMDimXpertOrientationGeoTol myOriTol;
double plusTol = 0.0;
double minusTol = 0.0;
double lowerTol = 0.0;
double upperTol = 0.0;
bool boolstatus;
vConfigNames = swDoc.ConfigurationManager.GetConfigurationNames();
if (vConfigNames != null)
{
    for (index = vConfigNames.GetLowerBound(0); index <= vConfigNames.GetUpperBound(0); index++)
    {
        configName = (string)vConfigNames[index];
        configObj = (SwDMConfiguration12)swDoc.ConfigurationManager.GetConfigurationByName(configName);
        if (configObj != null)
        {
            myDimXpertPart = configObj.DimXpertPart;
            if (myDimXpertPart != null)
            {
                vFeatures = myDimXpertPart.GetFeatures();
                if (vFeatures != null)
                {
                    Debug.Print(" ");
                    Debug.Print(myDimXpertPart.GetFeatureCount() + " DimXpert features:");
                    Debug.Print(" ");
                    for (featureIndex = vFeatures.GetLowerBound(0); featureIndex <= vFeatures.GetUpperBound(0); featureIndex++)
                    {
                        myFeature = (SwDMDimXpertFeature)vFeatures[featureIndex];
                        Debug.Print(featureIndex + " : " + "Feature name = " + myFeature.Name);
                        feattype = myFeature.type;
                        switch (feattype)
                        {
                            case swDmDimXpertFeatureType_e.swDmDimXpertFeature_Plane:
                            {
                                Debug.Print(" Type = Plane");
                                SwDMDimXpertPlaneFeature myPlaneFeature;
                                myPlaneFeature = (SwDMDimXpertPlaneFeature)myFeature;
                                boolstatus = myPlaneFeature.GetNominalPlane(X, Y, Z, i, j, k);
                                Debug.Print(" NomPlane(x,y,z,i,j,k): " + String.Format(X.ToString(), "0.0#####"));
                                break;
                            }

                            case swDmDimXpertFeatureType_e.swDmDimXpertFeature_BestfitPlane:
                            {
                                Debug.Print(" Type = Best fit plane");
                                SwDMDimXpertBestFitPlaneFeature myBestFitPlaneFeature;
                                myBestFitPlaneFeature = (SwDMDimXpertBestFitPlaneFeature)myFeature;
                                boolstatus = myBestFitPlaneFeature.GetNominalPlane(X, Y, Z, i, j, k);
                                Debug.Print(" NominalPlane(x,y,z,i,j,k): " + String.Format(X.ToString(), "0.0####"));
                                Debug.Print(" Number of sub-features: " + myBestFitPlaneFeature.GetSubFeatureCount());
                                if (myBestFitPlaneFeature.GetSubFeatureCount() > 0)
                                {
                                    Debug.Print(" Sub-features:");
                                    vObject = myBestFitPlaneFeature.GetSubFeatures();
                                    for (vObjectIndex = vObject.GetLowerBound(0); vObjectIndex <= vObject.GetUpperBound(0); vObjectIndex++)
                                    {
                                        tempFeature = (SwDMDimXpertFeature)vObject[vObjectIndex];
                                        Debug.Print(" " + tempFeature.Name);
                                    }
                                }
                            }
                        }
                    }
                }
            }
        }
    }
}

```

```

    }
}
break;
}

case swDmDimXpertFeatureType_e.swDmDimXpertFeature_Chamfer:
{
    Debug.Print(" Type = Chamfer");
    SwDMDimXpertChamferFeature myChamferFeature;
    myChamferFeature = (SwDMDimXpertChamferFeature)myFeature;
    Debug.Print(" Chamfer angle: " + myChamferFeature.Angle);
    Debug.Print(" Chamfer angle type (swDmDimXpertChamferAngleType_e): " + myChamferFeature.AngleType_e);
    Debug.Print(" Chamfer type (swDmDimXpertChamferType_e): " + myChamferFeature.ChamferType_e);
    Debug.Print(" Distance 1: " + myChamferFeature.Distance1);
    Debug.Print(" Distance 2: " + myChamferFeature.Distance2);
    break;
}

case swDmDimXpertFeatureType_e.swDmDimXpertFeature_CompoundClosedSlot3D:
{
    Debug.Print(" Type = Compound closed slot 3D");
    SwDMDimXpertCompoundClosedSlot3dFeature myClosedSlotFeature;
    myClosedSlotFeature = (SwDMDimXpertCompoundClosedSlot3dFeature)myFeature;
    tempFeature = myClosedSlotFeature.GetBottomFeature();
    if (tempFeature != null)
        Debug.Print(" Bottom feature: " + tempFeature.Name);
    endFeature1 = null;
    endFeature2 = null;
    planeFeature1 = null;
    planeFeature2 = null;
    boolstatus = myClosedSlotFeature.GetEndFeatures(endFeature1, endFeature2);
    if (endFeature1 != null & endFeature2 != null)
        Debug.Print(" End features: " + endFeature1.Name + ", " + endFeature2.Name);
    boolstatus = myClosedSlotFeature.GetNominalBottomPlane(X, Y, Z, i, j, k);
    Debug.Print(" Bottom plane(x,y,z,i,j,k): " + String.Format(X.ToString(), "0.0###"));
    boolstatus = myClosedSlotFeature.GetNominalClosedSlot(width, length, X, Y, Z, i, j, k);
    Debug.Print(" Closed slot(width,length,x,j,z,i,j,k,longi,longj,longk): " + String.Format(X.ToString(), "0.0###"));
    boolstatus = myClosedSlotFeature.GetNominalTopPlane(X, Y, Z, i, j, k);
    Debug.Print(" Top plane(x,y,z,i,j,k): " + String.Format(X.ToString(), "0.0#####"));
    boolstatus = myClosedSlotFeature.GetPlaneFeatures(planeFeature1, planeFeature2);
    if (planeFeature1 != null & planeFeature2 != null)
        Debug.Print(" Plane features: " + ((SwDMDimXpertFeature)planeFeature1).Name);
    Debug.Print(" Closed slot is blind and not through all: " + myClosedSlotFeature.IsBlind);
    break;
}

case swDmDimXpertFeatureType_e.swDmDimXpertFeature_CompoundHole:
{
    Debug.Print(" Type = Compound hole");
    SwDMDimXpertCompoundHoleFeature myHole;
    myHole = (SwDMDimXpertCompoundHoleFeature)myFeature;
    Debug.Print(" Hole is blind and not through all: " + myHole.Blind);
    SwDMDimXpertFeature bottomFeature;
    bottomFeature = myHole.GetBottomFeature();
    if (!bottomFeature == null)
        Debug.Print(" Bottom feature type (swDmDimXpertFeatureType_e) = " + bottomFeature.Type_e);
    SwDMDimXpertFeature refFeature;
    refFeature = myHole.GetReferenceFeature();
    if (!refFeature == null)
        Debug.Print(" Reference feature type (swDmDimXpertFeatureType_e) = " + refFeature.Type_e);
    if (myHole.GetSubFeatureCount() > 0)
    {
        vObject = myHole.GetSubFeatures();
        Debug.Print(" Sub-features: ");
        for (vObjectIndex = vObject.GetLowerBound(0); vObjectIndex <= vObject.GetUpperBound(0); vObjectIndex++)
        {
            tempFeature = (SwDMDimXpertFeature)vObject[vObjectIndex];
            Debug.Print(" " + tempFeature.Name);
        }
    }
    break;
}

case swDmDimXpertFeatureType_e.swDmDimXpertFeature_CompoundNotch:
{
    Debug.Print(" Type = Compound notch");
    SwDMDimXpertCompoundNotchFeature myNotchFeature;
    myNotchFeature = (SwDMDimXpertCompoundNotchFeature)myFeature;
    tempFeature = myNotchFeature.GetBottomFeature();
    if (tempFeature != null)
        Debug.Print(" Bottom feature: " + tempFeature.Name);
    endFeature1 = myNotchFeature.GetEndFeature();
    if (endFeature1 != null)
        Debug.Print(" End feature: " + endFeature1.Name);
    boolstatus = myNotchFeature.GetNominalBottomPlane(X, Y, Z, i, j, k);
    Debug.Print(" Bottom plane(x,y,z,i,j,k): " + String.Format(X.ToString(), "0.0###"));
    boolstatus = myNotchFeature.GetNominalNotch(width, length, X, Y, Z, i, j, k, longi, longj, longk);
    Debug.Print(" Notch(width,length,x,j,z,i,j,k,longi,longj,longk): " + String.Format(X.ToString(), "0.0#####"));
    tempFeature = myNotchFeature.GetOpenSideReferenceFeature();
    if (tempFeature != null)
        Debug.Print(" Open side reference feature: " + tempFeature.Name);
    tempFeature = myNotchFeature.GetTopReferenceFeature();
    if (tempFeature != null)
        Debug.Print(" Top reference feature: " + tempFeature.Name);
}

```

```

        planeFeature1 = null;
        planeFeature2 = null;
        boolstatus = myNotchFeature.GetPlaneFeatures(planeFeature1, planeFeature2);
        if (planeFeature1 != null & planeFeature2 != null)
            Debug.Print("Plane features: " + ((SwDMDimXpertFeature)planeFeature1).Name);
        Debug.Print("Notch is blind and not through all: " + myNotchFeature.Blind);
        break;
    }

case swDmDimXpertFeatureType_e.swDmDimXpertFeature_CompoundWidth:
{
    Debug.Print("Type = Compound width");
    SwDMDimXpertCompoundWidthFeature myWidthFeature;
    myWidthFeature = (SwDMDimXpertCompoundWidthFeature)myFeature;
    boolstatus = myWidthFeature.GetNominalCompoundWidth(width, X, Y, Z, i, j, k);
    Debug.Print("Compound width(width,x,j,z,i,j,k): " + String.Format(width.ToString(),
    boolstatus = myWidthFeature.GetNominalLongitude(i, j, k);
    Debug.Print("Nominal longitude(longi, longj, longk): " + String.Format(longi.To
    Debug.Print("Compound width is for a hole and not a pin: " + myWidthFeature.Inr
    break;
}

case swDmDimXpertFeatureType_e.swDmDimXpertFeature_Cone:
{
    Debug.Print("Type = Cone");
    SwDMDimXpertConeFeature myConeFeature;
    myConeFeature = (SwDMDimXpertConeFeature)myFeature;
    boolstatus = myConeFeature.GetNominalBottomPlane(X, Y, Z, i, j, k);
    Debug.Print("Nominal bottom plane(x,y,z,i,j,k): " + String.Format(X.ToString(),
    boolstatus = myConeFeature.GetNominalCone(angle, X, Y, Z, i, j, k);
    Debug.Print("Cone(angle,x,j,z,i,j,k): " + String.Format(angle.ToString(), "0.0#
    boolstatus = myConeFeature.GetNominalTopPlane(X, Y, Z, i, j, k);
    Debug.Print("Nominal top plane(x,y,z,i,j,k): " + String.Format(X.ToString(), '
    Debug.Print("Cone is a hole and not a pin: " + myConeFeature.Inner);
    break;
}

case swDmDimXpertFeatureType_e.swDmDimXpertFeature_Cylinder:
{
    Debug.Print("Type = Cylinder");
    SwDMDimXpertCylinderFeature myCylinder;
    myCylinder = (SwDMDimXpertCylinderFeature)myFeature;
    boolstatus = myCylinder.GetNominalTopPlane(X, Y, Z, i, j, k);
    Debug.Print("TopPlane(x,y,z,i,j,k): " + String.Format(X.ToString(), "0.0####")
    boolstatus = myCylinder.GetNominalCylinder(angle, X, Y, Z, i, j, k);
    Debug.Print("Cylinder(radius,x,j,z,i,j,k): " + String.Format(angle.ToString(),
    boolstatus = myCylinder.GetNominalBottomPlane(X, Y, Z, i, j, k);
    Debug.Print("BottomPlane(x,y,z,i,j,k): " + String.Format(X.ToString(), "0.0####
    bool isThreaded = false;
    string threadDesignation;
    double threadDepth = 0.0;
    threadDesignation = null;
    boolstatus = myCylinder.GetThread(isThreaded, threadDesignation, threadDepth);
    Debug.Print("Cylinder is threaded: " + isThreaded);
    if ((isThreaded))
    {
        Debug.Print("Cylinder thread designation: " + threadDesignation);
        Debug.Print("Cylinder thread depth: " + threadDepth);
    }
    Debug.Print("Cylinder is a hole and not a pin: " + myCylinder.Inner);
    break;
}

case swDmDimXpertFeatureType_e.swDmDimXpertFeature_Extrude:
{
    Debug.Print("Type = Extrude");
    SwDMDimXpertExtrudeFeature myExtrude;
    myExtrude = (SwDMDimXpertExtrudeFeature)myFeature;
    tempFeature = myExtrude.GetBottomFeature();
    if (tempFeature != null)
        Debug.Print("Bottom feature: " + tempFeature.Name);
    tempFeature = myExtrude.GetBottomBlends();
    if (tempFeature != null)
        Debug.Print("Bottom blends: " + tempFeature.Name);
    tempFeature = myExtrude.GetTopBlends();
    if (tempFeature != null)
        Debug.Print("Top blends: " + tempFeature.Name);
    tempFeature = myExtrude.GetReferenceFeature();
    if (tempFeature != null)
        Debug.Print("Reference feature: " + tempFeature.Name);
    Debug.Print("Number of sub-features: " + myExtrude.GetSubFeatureCount());
    if (myExtrude.GetSubFeatureCount() > 0)
    {
        Debug.Print("Sub-features:");
        vObject = myExtrude.GetSubFeatures();
        for (vObjectIndex = vObject.GetLowerBound(0); vObjectIndex <= vObject.GetUpperBo
        {
            tempFeature = (SwDMDimXpertFeature)vObject[vObjectIndex];
            Debug.Print(" " + tempFeature.Name);
        }
    }
    Debug.Print("Extrude is blind and not through all: " + myExtrude.Blind);
    break;
}

case swDmDimXpertFeatureType_e.swDmDimXpertFeature_Fillet:

```

```

{
    Debug.Print(" Type = Fillet");
    SwDMDimXpertFilletFeature myFillet;
    myFillet = (SwDMDimXpertFilletFeature)myFeature;
    Debug.Print(" Fillet is for a hole and not a pin: " + myFillet.Inner);
    Debug.Print(" Radius = " + myFillet.Radius);
    break;
}

case swDmDimXpertFeatureType_e.swDmDimXpertFeature_IntersectCircle:
{
    Debug.Print(" Type = Intersect circle");
    SwDMDimXpertIntersectCircleFeature myICircle;
    myICircle = (SwDMDimXpertIntersectCircleFeature)myFeature;
    tempFeature = myICircle.GetIntersectFeature();
    if (tempFeature != null)
        Debug.Print(" Intersect feature: " + tempFeature.Name);
    tempFeature = myICircle.GetPlaneFeature();
    if (tempFeature != null)
        Debug.Print(" Plane feature: " + tempFeature.Name);
    boolstatus = myICircle.GetNominalCircle(angle, X, Y, Z, i, j, k);
    Debug.Print(" Intersect circle(radius,x,y,z,i,j,k): " + String.Format(angle.ToString(), "0.0##"));
    break;
}

case swDmDimXpertFeatureType_e.swDmDimXpertFeature_IntersectLine:
{
    Debug.Print(" Type = Intersect line");
    SwDMDimXpertIntersectLineFeature myILine;
    myILine = (SwDMDimXpertIntersectLineFeature)myFeature;
    tempFeature = null;
    tempFeature1 = null;
    boolstatus = myILine.GetPlaneFeatures(tempFeature, tempFeature1);
    if (tempFeature != null & tempFeature1 != null)
    {
        Debug.Print(" Plane feature 1: " + tempFeature.Name);
        Debug.Print(" Plane feature 2: " + tempFeature1.Name);
    }
    boolstatus = myILine.GetNominalLine(X, Y, Z, i, j, k);
    Debug.Print(" Intersect line(x,y,z,i,j,k): " + String.Format(X.ToString(), "0.0##"));
    break;
}

case swDmDimXpertFeatureType_e.swDmDimXpertFeature_IntersectPlane:
{
    Debug.Print(" Type = Intersect plane");
    SwDMDimXpertIntersectPlaneFeature myIPlane;
    myIPlane = (SwDMDimXpertIntersectPlaneFeature)myFeature;
    tempFeature = null;
    tempFeature1 = null;
    boolstatus = myIPlane.GetFeatures(tempFeature, tempFeature1);
    if (tempFeature != null & tempFeature1 != null)
    {
        Debug.Print(" Feature 1: " + tempFeature.Name);
        Debug.Print(" Feature 2: " + tempFeature1.Name);
    }
    boolstatus = myIPlane.GetNominalPlane(X, Y, Z, i, j, k);
    Debug.Print(" Intersect plane(x,y,z,i,j,k): " + String.Format(X.ToString(), "0.0#"));
    break;
}

case swDmDimXpertFeatureType_e.swDmDimXpertFeature_IntersectPoint:
{
    Debug.Print(" Type = Intersect point");
    SwDMDimXpertIntersectPointFeature myIPoint;
    myIPoint = (SwDMDimXpertIntersectPointFeature)myFeature;
    tempFeature = null;
    tempFeature1 = null;
    boolstatus = myIPoint.GetFeatures(tempFeature, tempFeature1);
    if (tempFeature != null & tempFeature1 != null)
    {
        Debug.Print(" Feature 1: " + tempFeature.Name);
        Debug.Print(" Feature 2: " + tempFeature1.Name);
    }
    boolstatus = myIPoint.GetNominalPoint(X, Y, Z);
    boolstatus = myIPoint.GetNominalVector(i, j, k);
    Debug.Print(" Intersect point(x,y,z,i,j,k): " + String.Format(X.ToString(), "0.0#"));
    break;
}

case swDmDimXpertFeatureType_e.swDmDimXpertFeature_Pattern:
{
    Debug.Print(" Type = Pattern");
    SwDMDimXpertPatternFeature myPattern;
    myPattern = (SwDMDimXpertPatternFeature)myFeature;
    Debug.Print(" Number of sub-features: " + myPattern.GetSubFeatureCount());
    if (myPattern.GetSubFeatureCount() > 0)
    {
        Debug.Print(" Sub-features:");
        vObject = myPattern.GetSubFeatures();
        for (vObjectIndex = vObject.GetLowerBound(0); vObjectIndex <= vObject.GetUpperBound(0); vObjectIndex++)
        {
            tempFeature = (SwDMDimXpertFeature)vObject[vObjectIndex];
            Debug.Print(" " + tempFeature.Name);
        }
    }
}

```

```

        break;
    }

case swDmDimXpertFeatureType_e.swDmDimXpertFeature_Sphere:
{
    Debug.Print(" Type = Sphere");
    SwDmDimXpertSphereFeature mySphere;
    mySphere = (SwDmDimXpertSphereFeature)myFeature;
    boolstatus = mySphere.GetNominalSphere(angle, X, Y, Z);
    Debug.Print(" Sphere(radius,x,j,z): " + String.Format(angle.ToString(), "0.0####"));
    Debug.Print(" Sphere is for a hole and not a pin: " + mySphere.Inner);
    break;
}

case swDmDimXpertFeatureType_e.swDmDimXpertFeature_Surface:
{
    Debug.Print(" Type = Surface");
    break;
}

default:
{
    Debug.Print(" Type = <unknown>");
    break;
}
}
Debug.Print(" CAD identifier count = " + myFeature.GetCadIdentifierCount());
if (myFeature.GetCadIdentifierCount() > 0)
{
    Debug.Print(" CAD Identifiers:");
    vObject = myFeature.GetCadIdentifiers();
    for (vObjectIndex = vObject.GetLowerBound(0); vObjectIndex <= vObject.GetUpperBound(0); vOb;
        Debug.Print(" " + vObject[vObjectIndex]);
    }
    Debug.Print(" Suppressed = " + myFeature.IsSuppressed());
    Debug.Print(" ");
}
}

myBlockTolerance = myDimXpertPart.GetBlockTolerances();
Debug.Print(" DimXpert block tolerance settings:");
Debug.Print(" ");
double linear1 = 0.0;
double linear2 = 0.0;
double linear3 = 0.0;
double angular = 0.0;
int linear1Prec = 0;
int linear2Prec = 0;
int linear3Prec = 0;
bltType = myBlockTolerance.type;
switch (bltType)
{
    case swDmDimXpertBlockToleranceType_e.swDmDimXpertBlockToleranceType_ASMEInch:
    {
        boolstatus = myBlockTolerance.GetToleranceValues(linear1, linear1Prec, linear2, linear2Prec,
            Debug.Print(" ASMEInch Dimension 1 tolerance: " + linear1);
            Debug.Print(" ASMEInch Dimension 1 tolerance precision: " + linear1Prec);
            Debug.Print(" ASMEInch Dimension 2 tolerance: " + linear2);
            Debug.Print(" ASMEInch Dimension 2 tolerance precision: " + linear2Prec);
            Debug.Print(" ASMEInch Dimension 3 tolerance: " + linear3);
            Debug.Print(" ASMEInch Dimension 3 tolerance precision: " + linear3Prec);
            Debug.Print(" ASMEInch Angular tolerance: " + angular);
            break;
        }

    case swDmDimXpertBlockToleranceType_e.swDmDimXpertBlockToleranceType_ISO2768:
    {
        swDmDimXpertISO2768PartType_e ISO2768Type = 0;
        boolstatus = myBlockTolerance.GetISO2768PartType(ISO2768Type);
        Debug.Print(" ISO 2768 part type (swDmDimXpertISO2768PartType_e): " + ISO2768Type);
        break;
    }

    case swDmDimXpertBlockToleranceType_e.swDmDimXpertBlockToleranceType_unknown:
    {
        Debug.Print(" Block tolerance unknown");
        break;
    }
}

vAnnotations = myDimXpertPart.GetAnnotations();
if (vAnnotations != null)
{
    Debug.Print(" ");
    Debug.Print(myDimXpertPart.GetAnnotationCount() + " DimXpert annotations:");
    Debug.Print(" ");
    for (annoIndex = vAnnotations.GetLowerBound(0); annoIndex <= vAnnotations.GetUpperBound(0); annoInde
    {
        myDimTol = null;
        myGeoTol = null;
        myOriTol = null;
        myAnnotation = (SwDmDimXpertAnnotation)vAnnotations[annoIndex];
        Debug.Print(annoIndex + " : " + "Annotation name = " + myAnnotation.Name);
        annotype = myAnnotation.type;
        switch (annotype)
        {
            case swDmDimXpertAnnotationType_e.swDmDimXpertDatum:

```

```

{
    Debug.Print(" Type = Datum");
    swDmDimXpertDatum myDatum;
    myDatum = (swDmDimXpertDatum)myAnnotation;
    Debug.Print(" ID = " + myDatum.Identifier);
    break;
}

case swDmDimXpertAnnotationType_e.swDmDimXpertDimTol_AngleBetween:
{
    Debug.Print(" Type = Angle-between dimension tolerance");
    SwDMDimXpertAngleBetweenDimTol myAngBetDimTol;
    myAngBetDimTol = (SwDMDimXpertAngleBetweenDimTol)myAnnotation;
    boolstatus = myAngBetDimTol.GetDirectionVector(i, j, k);
    Debug.Print(" DirectionVector(i,j,k): " + i + ", " + j + ", " + k);
    Debug.Print(" Feature of origin: " + myAngBetDimTol.OriginFeature.Name);
    Debug.Print(" Calculate supplement angle? " + myAngBetDimTol.Supplement);
    myDimTol = (SwDMDimXpertDimensionTolerance)myAnnotation;
    break;
}

case swDmDimXpertAnnotationType_e.swDmDimXpertDimTol_ChamferDimension:
{
    Debug.Print(" Type = Chamfer dimension tolerance");
    myDimTol = (SwDMDimXpertDimensionTolerance)myAnnotation;
    SwDMDimXpertChamferDimTol myChamferDimTol;
    myChamferDimTol = (SwDMDimXpertChamferDimTol)myAnnotation;
    Debug.Print(" Chamfer dimension type (swDmDimXpertChamferDimensionType_e): " + n
    break;
}

case swDmDimXpertAnnotationType_e.swDmDimXpertDimTol_CompositeDistanceBetween:
{
    Debug.Print(" Type = Composite distance-between dimension tolerance");
    myDimTol = (SwDMDimXpertDimensionTolerance)myAnnotation;
    SwDMDimXpertCompositeDistanceBetweenDimTol myCompDistBetDimTol;
    myCompDistBetDimTol = (SwDMDimXpertCompositeDistanceBetweenDimTol)myAnnotation;
    boolstatus = myCompDistBetDimTol.GetDirectionVector(i, j, k);
    Debug.Print(" Direction vector(i,j,k): " + i + ", " + j + ", " + k);
    tempFeature1 = null;
    boolstatus = myCompDistBetDimTol.GetFeature(tempFeature1, fosUsage);
    if (tempFeature1 != null)
    {
        Debug.Print(" Composite distance-
        Debug.Print(" Feature of size usage (swDmDimXpertDistanceFosUsage_e): " + fc
        }
        boolstatus = myCompDistBetDimTol.GetIntraFeature(tempFeature1, fosUsage);
        if (tempFeature1 != null)
        {
            Debug.Print(" Feature-locating feature: " + tempFeature1.Name);
            Debug.Print(" Feature of size usage for feature-
            locating(swDmDimXpertDistanceFosUsage_e): " + fosUsage);
        }
        boolstatus = myCompDistBetDimTol.GetOriginFeature(tempFeature1, fosUsage);
        if (tempFeature1 != null)
        {
            Debug.Print(" Feature of origin for this dimension tolerance: " + tempFeatur
            Debug.Print(" Feature of size usage (swDmDimXpertDistanceFosUsage_e): " + fc
        }
        Debug.Print(" Type of dimension tolerance for feature-
        locating (swDmDimXpertDimensionToleranceType_e): " + myCompDistBetDimTol.DimensionTypeIntraFeature);
        Debug.Print(" Tolerance for feature-
        locating: " + myCompDistBetDimTol.GetNominalValueIntraFeature());
        boolstatus = myCompDistBetDimTol.GetPlusAndMinusToleranceIntraFeature(plusTol, minus
        Debug.Print(" Plus / Minus limit for feature-
        locating: " + plusTol + " / " + minusTol);
        boolstatus = myCompDistBetDimTol.GetUpperAndLowerLimitIntraFeature(upperTol, lowerTc
        Debug.Print(" Upper / Lower limit for feature-
        locating: " + upperTol + " / " + lowerTol);
        Debug.Print(" Block tolerance precision for feature-
        locating: " + myCompDistBetDimTol.BlockToleranceDecimalPlacesIntraFeature);
        Debug.Print(" Limits and fits code for feature-
        locating: " + myCompDistBetDimTol.LimitsAndFitsCodeIntraFeature);
        break;
    }
}

case swDmDimXpertAnnotationType_e.swDmDimXpertDimTol_ConeAngle:
{
    Debug.Print(" Type = Cone angle dimension tolerance");
    myDimTol = (SwDMDimXpertDimensionTolerance)myAnnotation;
    break;
}

case swDmDimXpertAnnotationType_e.swDmDimXpertDimTol_CounterBore:
{
    Debug.Print(" Type = Counterbore dimension tolerance");
    myDimTol = (SwDMDimXpertDimensionTolerance)myAnnotation;
    SwDMDimXpertCounterBoreDimTol myCounterBoreDimTol;
    myCounterBoreDimTol = (SwDMDimXpertCounterBoreDimTol)myAnnotation;
    tempFeature = myCounterBoreDimTol.ReferenceFeature;
    if (tempFeature != null)
        Debug.Print(" Reference Feature for this dimension tolerance: " + tempFeatur
    break;
}
}

```

```

case swDmDimXpertAnnotationType_e.swDmDimXpertDimTol_CounterSinkAngle:
{
    Debug.Print(" Type = Countersink angle dimension tolerance");
    myDimTol = (SwDMDimXpertDimensionTolerance)myAnnotation;
    SwDMDimXpertCounterSinkAngleDimTol myCounterSinkAngleDimTol;
    myCounterSinkAngleDimTol = (SwDMDimXpertCounterSinkAngleDimTol)myAnnotation;
    tempFeature = myCounterSinkAngleDimTol.ReferenceFeature;
    if (tempFeature != null)
        Debug.Print(" Reference Feature for this dimension tolerance: " + tempFeatur
    break;
}

case swDmDimXpertAnnotationType_e.swDmDimXpertDimTol_CounterSinkDiameter:
{
    Debug.Print(" Type = Countersink diameter dimension tolerance");
    myDimTol = (SwDMDimXpertDimensionTolerance)myAnnotation;
    SwDMDimXpertCounterSinkDiameterDimTol myCounterSinkDiameterDimTol;
    myCounterSinkDiameterDimTol = (SwDMDimXpertCounterSinkDiameterDimTol)myAnnotation;
    tempFeature = myCounterSinkDiameterDimTol.ReferenceFeature;
    if (tempFeature != null)
        Debug.Print(" Reference Feature for this dimension tolerance: " + tempFeatur
    break;
}

case swDmDimXpertAnnotationType_e.swDmDimXpertDimTol_Depth:
{
    Debug.Print(" Type = Depth dimension tolerance");
    myDimTol = (SwDMDimXpertDimensionTolerance)myAnnotation;
    SwDMDimXpertDepthDimTol myDepthDimTol;
    myDepthDimTol = (SwDMDimXpertDepthDimTol)myAnnotation;
    Debug.Print(" Tolerance is for thread depth: " + myDepthDimTol.IsThreadDepth());
    tempFeature = myDepthDimTol.ReferenceFeature;
    if (tempFeature != null)
        Debug.Print(" Reference Feature for this dimension tolerance: " + tempFeatur
    break;
}

case swDmDimXpertAnnotationType_e.swDmDimXpertDimTol_Diameter:
{
    Debug.Print(" Type = Diameter dimension tolerance");
    myDimTol = (SwDMDimXpertDimensionTolerance)myAnnotation;
    SwDMDimXpertDiameterDimTol myDiameterDimTol;
    myDiameterDimTol = (SwDMDimXpertDiameterDimTol)myAnnotation;
    Debug.Print(" Pattern treatment type (swDmDimXpertPatternTreatmentType_e): " + n
    break;
}

case swDmDimXpertAnnotationType_e.swDmDimXpertDimTol_DistanceBetween:
{
    Debug.Print(" Type = Distance-between dimension tolerance");
    myDimTol = (SwDMDimXpertDimensionTolerance)myAnnotation;
    SwDMDimXpertDistanceBetweenDimTol myDistBetDimTol;

    myDistBetDimTol = (SwDMDimXpertDistanceBetweenDimTol)myAnnotation;
    boolstatus = myDistBetDimTol.GetDirectionVector(i, j, k);
    Debug.Print(" Direction vector(i,j,k): " + i + ", " + j + ", " + k);
    tempFeature1 = null;
    boolstatus = myDistBetDimTol.GetFeature(tempFeature1, fosUsage);
    if (tempFeature1 != null)
    {
        Debug.Print(" Distance-
between dimension tolerance feature: " + tempFeature1.Name);
        Debug.Print(" Feature of size usage (swDmDimXpertDistanceFosUsage_e): " + f
    }
    boolstatus = myDistBetDimTol.GetOriginFeature(tempFeature1, fosUsage);
    if (tempFeature1 != null)
    {
        Debug.Print(" Feature of origin for this dimension tolerance: " + tempFeatur
        Debug.Print(" Feature of size usage(swDmDimXpertDistanceFosUsage_e): " + fc
    }
    Debug.Print(" Pattern treatment type (swDmDimXpertPatternTreatmentType_e): " + n
    break;
}

case swDmDimXpertAnnotationType_e.swDmDimXpertDimTol_Length:
{
    Debug.Print(" Type = Length dimension tolerance");
    myDimTol = (SwDMDimXpertDimensionTolerance)myAnnotation;
    break;
}

case swDmDimXpertAnnotationType_e.swDmDimXpertDimTol_Radius:
{
    Debug.Print(" Type = Radius dimension tolerance");
    myDimTol = (SwDMDimXpertDimensionTolerance)myAnnotation;
    SwDMDimXpertRadiusDimTol myRadiusDimTol;
    myRadiusDimTol = (SwDMDimXpertRadiusDimTol)myAnnotation;
    Debug.Print(" Pattern treatment type (swDmDimXpertPatternTreatmentType_e): " + n
    break;
}

case swDmDimXpertAnnotationType_e.swDmDimXpertDimTol_Width:
{
    Debug.Print(" Type = Width dimension tolerance");
    myDimTol = (SwDMDimXpertDimensionTolerance)myAnnotation;
    break;
}

```

```

    }

case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_Angularity:
{
    Debug.Print(" " + annoIndex + " : Type = Angularity geometric tolerance");
    myGeoTol = (SwDMDimXpertGeometricTolerance)myAnnotation;
    myOriTol = (SwDMDimXpertOrientationGeoTol)myAnnotation;
    break;
}

case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_Circularity:
{
    Debug.Print(" Type = Circularity geometric tolerance");
    myGeoTol = (SwDMDimXpertGeometricTolerance)myAnnotation;
    break;
}

case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_CircularRunout:
{
    Debug.Print(" Type = Circular runout geometric tolerance");
    myGeoTol = (SwDMDimXpertGeometricTolerance)myAnnotation;
    break;
}

case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_CompositeLineProfile:
{
    Debug.Print(" Type = Composite line profile geometric tolerance");
    myGeoTol = (SwDMDimXpertGeometricTolerance)myAnnotation;
    SwDMDimXpertCompositeLineProfileGeoTol myCompLineProfileGeoTol;
    myCompLineProfileGeoTol = (SwDMDimXpertCompositeLineProfileGeoTol)myAnnotation;
    boolstatus = myCompLineProfileGeoTol.GetPlanarZoneVector(i, j, k);
    Debug.Print(" Planar zone vector(i,j,k): " + i + ", " + j + ", " + k);
    Debug.Print(" Outer tolerance value: " + myCompLineProfileGeoTol.OuterToleranceValue);
    Debug.Print(" Outer tolerance value in the lower tier: " + myCompLineProfileGeoTol.OuterToleranceValueLowerTier);
    Debug.Print(" Primary datum repeated in lower tier: " + myCompLineProfileGeoTol.PrimaryDatumRepeatedInLowerTier);
    Debug.Print(" Secondary datum repeated in lower tier: " + myCompLineProfileGeoTol.SecondaryDatumRepeatedInLowerTier);
    Debug.Print(" Tertiary datum repeated in lower tier: " + myCompLineProfileGeoTol.TertiaryDatumRepeatedInLowerTier);
    Debug.Print(" Tolerance in lower tier: " + myCompLineProfileGeoTol.ToleranceLowerTier);
    break;
}

case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_CompositePosition:
{
    Debug.Print(" Type = Composite position geometric tolerance");
    myGeoTol = (SwDMDimXpertGeometricTolerance)myAnnotation;
    SwDMDimXpertCompositePositionGeoTol myCompPosGeoTol;
    myCompPosGeoTol = (SwDMDimXpertCompositePositionGeoTol)myAnnotation;
    boolstatus = myCompPosGeoTol.GetMaxTolerance(isMax, Tolerance);
    Debug.Print(" Maximum tolerance is set: " + isMax);
    if (isMax)
        Debug.Print(" Maximum tolerance: " + Tolerance);
    boolstatus = myCompPosGeoTol.GetMaxToleranceLowerTier(isMax, Tolerance);
    Debug.Print(" Maximum tolerance is set in lower tier: " + isMax);
    if (isMax)
        Debug.Print(" Maximum tolerance in lower tier: " + Tolerance);
    boolstatus = myCompPosGeoTol.GetPlanarZoneVector(i, j, k);
    Debug.Print(" Planar zone vector(i,j,k): " + i + ", " + j + ", " + k);
    boolstatus = myCompPosGeoTol.GetProjectedZone(Enabled, Value);
    Debug.Print(" Projected zone is in force: " + Enabled);
    if (Enabled)
        Debug.Print("Value: " + Value);
    Debug.Print(" Material condition modifier (swDmDimXpertMaterialConditionModifier)");
    Debug.Print(" Material condition modifier in lower tier (swDmDimXpertMaterialConditionModifier)");
    Debug.Print(" Primary datum repeated in lower tier: " + myCompPosGeoTol.PrimaryDatumRepeatedInLowerTier);
    Debug.Print(" Secondary datum repeated in lower tier: " + myCompPosGeoTol.SecondaryDatumRepeatedInLowerTier);
    Debug.Print(" Tertiary datum repeated in lower tier: " + myCompPosGeoTol.TertiaryDatumRepeatedInLowerTier);
    Debug.Print(" Tolerance in lower tier: " + myCompPosGeoTol.ToleranceLowerTier);
    Debug.Print(" Position zone type (swDmDimXpertPositionZoneType_e): " + myCompPosGeoTol.PositionZoneType);
    break;
}

case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_CompositeSurfaceProfile:
{
    Debug.Print(" Type = Composite surface profile geometric tolerance");
    myGeoTol = (SwDMDimXpertGeometricTolerance)myAnnotation;
    SwDMDimXpertCompositeSurfaceProfileGeoTol myCompSurfProfGeoTol;
    myCompSurfProfGeoTol = (SwDMDimXpertCompositeSurfaceProfileGeoTol)myAnnotation;
    Debug.Print(" Outer tolerance: " + myCompSurfProfGeoTol.OuterToleranceValue);
    Debug.Print(" Outer tolerance in lower tier: " + myCompSurfProfGeoTol.OuterToleranceValueLowerTier);
    Debug.Print(" Primary datum repeated in lower tier: " + myCompSurfProfGeoTol.PrimaryDatumRepeatedInLowerTier);
    Debug.Print(" Secondary datum repeated in lower tier: " + myCompSurfProfGeoTol.SecondaryDatumRepeatedInLowerTier);
    Debug.Print(" Tertiary datum repeated in lower tier: " + myCompSurfProfGeoTol.TertiaryDatumRepeatedInLowerTier);
    Debug.Print(" Tolerance in lower tier: " + myCompSurfProfGeoTol.ToleranceLowerTier);
    break;
}

case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_Concentricity:
{
    Debug.Print(" Type = Concentricity geometric tolerance");
    myGeoTol = (SwDMDimXpertGeometricTolerance)myAnnotation;
    SwDMDimXpertConcentricityGeoTol myConcentricityGeoTol;
    myConcentricityGeoTol = (SwDMDimXpertConcentricityGeoTol)myAnnotation;
    boolstatus = myConcentricityGeoTol.GetMaxTolerance(isMax, Tolerance);
    Debug.Print(" Maximum tolerance is set: " + isMax);
    if (isMax)
        Debug.Print(" Maximum tolerance: " + Tolerance);
}

```

```

        Debug.Print("      Material condition modifier (swDmDimXpertMaterialConditionModifier
        break;
    }
}

case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_Cylindricity:
{
    Debug.Print("  Type = Cylindricity geometric tolerance");
    myGeoTol = (SwDMDimXpertGeometricTolerance)myAnnotation;
    break;
}

case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_Flatness:
{
    Debug.Print("  Type = Flatness geometric tolerance");
    myGeoTol = (SwDMDimXpertGeometricTolerance)myAnnotation;
    SwDMDimXpertFlatnessGeoTol myFlatnessGeoTol;
    myFlatnessGeoTol = (SwDMDimXpertFlatnessGeoTol)myAnnotation;
    boolstatus = myFlatnessGeoTol.GetPerUnitArea(Enabled, length, width, i, j, k);
    Debug.Print("      Per unit area(enabled,width,length,i,j,k): " + Enabled + ", " + St
    break;
}

case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_LineProfile:
{
    Debug.Print("  Type = Line profile geometric tolerance");
    myGeoTol = (SwDMDimXpertGeometricTolerance)myAnnotation;
    SwDMDimXpertLineProfileGeoTol myLineProfileGeoTol;
    myLineProfileGeoTol = (SwDMDimXpertLineProfileGeoTol)myAnnotation;
    boolstatus = myLineProfileGeoTol.GetPlanarZoneVector(i, j, k);
    Debug.Print("      Planar zone vector(i,j,k): " + i + ", " + j + ", " + k);
    Debug.Print("      Outer tolerance value: " + myLineProfileGeoTol.OuterToleranceValue
    break;
}

case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_Parallelism:
{
    Debug.Print("  Type = Parallelism geometric tolerance");
    myGeoTol = (SwDMDimXpertGeometricTolerance)myAnnotation;
    myOriTol = (SwDMDimXpertOrientationGeoTol)myAnnotation;
    break;
}

case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_Perpendicularity:
{
    Debug.Print("  Type = Perpendicularity geometric tolerance");
    myGeoTol = (SwDMDimXpertGeometricTolerance)myAnnotation;
    myOriTol = (SwDMDimXpertOrientationGeoTol)myAnnotation;
    break;
}

case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_Position:
{
    Debug.Print("  Type = Position geometric tolerance");
    myGeoTol = (SwDMDimXpertGeometricTolerance)myAnnotation;
    SwDMDimXpertPositionGeoTol myPosGeoTol;
    myPosGeoTol = (SwDMDimXpertPositionGeoTol)myAnnotation;
    boolstatus = myPosGeoTol.GetMaxTolerance(isMax, Tolerance);
    Debug.Print("      Maximum tolerance is set: " + isMax);
    if (isMax)
        Debug.Print("      Maximum tolerance: " + Tolerance);
    boolstatus = myPosGeoTol.GetPlanarZoneVector(i, j, k);
    Debug.Print("      Planar zone vector(i,j,k): " + i + ", " + j + ", " + k);
    boolstatus = myPosGeoTol.GetProjectedZone(Enabled, Value);
    Debug.Print("      Projected zone is in force: " + Enabled);
    if (Enabled)
        Debug.Print("      Value: " + Value);
    Debug.Print("      Material condition modifier (swDmDimXpertMaterialConditionModifier
    Debug.Print("      Position zone type (swDmDimXpertPositionZoneType_e): " + myPosGeoT
    break;
}

case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_Straightness:
{
    Debug.Print("  Type = Straightness geometric tolerance");
    myGeoTol = (SwDMDimXpertGeometricTolerance)myAnnotation;
    SwDMDimXpertStraightnessGeoTol myStraightnessGeoTol;
    myStraightnessGeoTol = (SwDMDimXpertStraightnessGeoTol)myAnnotation;
    boolstatus = myStraightnessGeoTol.GetMaxTolerance(isMax, Tolerance);
    Debug.Print("      Maximum tolerance is set: " + isMax);
    if (isMax)
        Debug.Print("      Maximum tolerance: " + Tolerance);
    boolstatus = myStraightnessGeoTol.GetPerUnitLength(Enabled, Distance);
    Debug.Print("      Per unit length(enabled,distance): " + Enabled + ", " + String.For
    boolstatus = myStraightnessGeoTol.GetPlanarZoneVector(i, j, k);
    Debug.Print("      Planar zone vector(i,j,k): " + i + ", " + j + ", " + k);
    Debug.Print("      Material condition modifier (swDmDimXpertMaterialConditionModifier
    Debug.Print("      Position zone type (swDmDimXpertStraightnessZoneType_e): " + myStr
    break;
}

case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_SurfaceProfile:
{
    Debug.Print("  Type = SurfaceProfile geometric tolerance");
    myGeoTol = (SwDMDimXpertGeometricTolerance)myAnnotation;
    SwDMDimXpertSurfaceProfileGeoTol mySurfProfGeoTol;
    mySurfProfGeoTol = (SwDMDimXpertSurfaceProfileGeoTol)myAnnotation;

```

```

        Debug.Print("    Outer tolerance: " + mySurfProfGeoTol.OuterToleranceValue);
        break;
    }
}

case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_Symmetry:
{
    Debug.Print("    Type = Symmetry geometric tolerance");
    myGeoTol = (SwDmDimXpertGeometricTolerance)myAnnotation;
    SwDmDimXpertSymmetryGeoTol mySymmetryGeoTol;
    mySymmetryGeoTol = (SwDmDimXpertSymmetryGeoTol)myAnnotation;
    boolstatus = mySymmetryGeoTol.GetMaxTolerance(isMax, Tolerance);
    Debug.Print("    Maximum tolerance is set: " + isMax);
    Debug.Print("    Maximum tolerance: " + Tolerance);
    boolstatus = mySymmetryGeoTol.GetZoneDirection(i, j, k);
    Debug.Print("    Zone direction(i,j,k): " + i + ", " + j + ", " + k);
    modifier = mySymmetryGeoTol.Modifier;
    Debug.Print("    Modifier (swDmDimXpertMaterialConditionModifier_e): " + modifier);
    break;
}

case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_Tangency:
{
    Debug.Print("    Type = Tangency geometric tolerance");
    SwDmDimXpertTangencyGeoTol myTangencyGeoTol;
    myTangencyGeoTol = (SwDmDimXpertTangencyGeoTol)myAnnotation;
    tempFeature = null;
    boolstatus = myTangencyGeoTol.GetOriginFeature(tempFeature);
    if (tempFeature != null)
        Debug.Print("    Feature of origin for this geometric tolerance: " + tempFeature.Name);
    break;
}

case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_TotalRunout:
{
    Debug.Print("    Type = Total runout geometric tolerance");
    myGeoTol = (SwDmDimXpertGeometricTolerance)myAnnotation;
    break;
}

default:
{
    Debug.Print("    Type = <unknown>");
    break;
}
}

// Annotation Information
Debug.Print("    Annotation Information:");
Debug.Print("    CAD identifier count = " + myAnnotation.GetCadIdentifierCount());
if (myAnnotation.GetCadIdentifierCount() > 0)
{
    Debug.Print("    CAD Identifiers:");
    vObject = myAnnotation.GetCadIdentifiers();
    for (vObjectIndex = vObject.GetLowerBound(0); vObjectIndex <= vObject.GetUpperBound(0); vObjectIndex++)
        Debug.Print("        " + vObject[vObjectIndex]);
}
Debug.Print("    Free state = " + myAnnotation.IsFreeState());
Debug.Print("    Statistical = " + myAnnotation.IsStatistical());
Debug.Print("    Suppressed = " + myAnnotation.IsSuppressed());
Debug.Print("    To be inspected = " + myAnnotation.IsToBeInspected());
if ((myAnnotation.Feature == null))
    Debug.Print("    Is not associated with a feature");
else
    Debug.Print("    Is associated with a feature");

// Geometric Tolerance Information for Orientation Dimensions
if (myOriTol != null)
{
    Debug.Print("    Orientation Geometric Tolerance Information:");
    boolstatus = myOriTol.GetMaxTolerance(isMax, Tolerance);
    Debug.Print("    Maximum tolerance is set: " + isMax);
    if (isMax)
        Debug.Print("    Maximum tolerance: " + Tolerance);
    boolstatus = myOriTol.GetPlanarZoneVector(i, j, k);
    Debug.Print("    Planar zone vector(i,j,k): " + i + ", " + j + ", " + k);
    boolstatus = myOriTol.GetProjectedZone(Enabled, Value);
    Debug.Print("    Projected zone is in force: " + Enabled);
    if (Enabled)
        Debug.Print("    Value: " + Value);
    Debug.Print("    Material condition modifier (swDmDimXpertMaterialConditionModifier_e): " + myOriTol.Modifier);
    Debug.Print("    Position zone type (swDmDimXpertOrientationZoneType_e): " + myOriTol.ZoneType);
}

// Geometric Tolerance Information
if (myGeoTol != null)
{
    Debug.Print("    Geometric Tolerance Information:");
    Debug.Print("    Number of primary datums: " + myGeoTol.GetPrimaryDatumCount());
    if (myGeoTol.GetPrimaryDatumCount() > 0)
    {
        Debug.Print("    Primary Datum modifiers:");
        vObject = myGeoTol.GetPrimaryDatumModifiers();
        for (vObjectIndex = vObject.GetLowerBound(0); vObjectIndex <= vObject.GetUpperBound(0); vObjectIndex++)
        {
            modifier = (swDmDimXpertMaterialConditionModifier_e)vObject[vObjectIndex];
            Debug.Print("        Primary datum modifier (swDmDimXpertMaterialConditionModifier_e): " + modifier);
        }
    }
}

```

```

        }
        Debug.Print("    Primary datums:");
        vObject = myGeoTol.GetPrimaryDatums();
        for (vObjectIndex = vObject.GetLowerBound(0); vObjectIndex <= vObject.GetUpperBound(0); )
        {
            datum = (swDmDimXpertDatum)vObject[vObjectIndex];
            Debug.Print("        Primary datum : " + datum.Identifier);
        }
    }
    Debug.Print("    Number of secondary datums: " + myGeoTol.GetSecondaryDatumCount());
    if (myGeoTol.GetSecondaryDatumCount() > 0)
    {
        Debug.Print("    Secondary Datum modifiers:");
        vObject = myGeoTol.GetSecondaryDatumModifiers();
        for (vObjectIndex = vObject.GetLowerBound(0); vObjectIndex <= vObject.GetUpperBound(0); )
        {
            modifier = (swDmDimXpertMaterialConditionModifier_e)vObject[vObjectIndex];
            Debug.Print("        Secondary datum modifier (swDmDimXpertMaterialConditionModifier_");
        }
        Debug.Print("    Secondary datums:");
        vObject = myGeoTol.GetSecondaryDatums();
        for (vObjectIndex = vObject.GetLowerBound(0); vObjectIndex <= vObject.GetUpperBound(0); )
        {
            datum = (swDmDimXpertDatum)vObject[vObjectIndex];
            Debug.Print("        Secondary datum : " + datum.Identifier);
        }
    }
    Debug.Print("    Number of tertiary datums: " + myGeoTol.GetTertiaryDatumCount());
    if (myGeoTol.GetTertiaryDatumCount() > 0)
    {
        Debug.Print("    Tertiary Datum modifiers:");
        vObject = myGeoTol.GetTertiaryDatumModifiers();
        for (vObjectIndex = vObject.GetLowerBound(0); vObjectIndex <= vObject.GetUpperBound(0); )
        {
            modifier = (swDmDimXpertMaterialConditionModifier_e)vObject[vObjectIndex];
            Debug.Print("        Tertiary datum modifier (swDmDimXpertMaterialConditionModifier_");
        }
        Debug.Print("    Tertiary datums:");
        vObject = myGeoTol.GetTertiaryDatums();
        for (vObjectIndex = vObject.GetLowerBound(0); vObjectIndex <= vObject.GetUpperBound(0); )
        {
            datum = (swDmDimXpertDatum)vObject[vObjectIndex];
            Debug.Print("        Tertiary datum : " + datum.Identifier);
        }
    }
    Debug.Print("    Geometric Tolerance = " + myGeoTol.Tolerance);
}

// Dimension Tolerance Information
if (myDimTol != null)
{
    Debug.Print("    Dimension Tolerance Information:");
    Debug.Print("        Dimension tolerance type (swDmDimXpertDimensionToleranceType_e)= " + myDimTol.DimensionToleranceType);
    Debug.Print("        Dimension tolerance = " + myDimTol.GetNominalValue());
    boolstatus = myDimTol.GetPlusAndMinusTolerance(plusTol, minusTol);
    Debug.Print("        Plus / Minus   " + plusTol + " / " + minusTol);
    boolstatus = myDimTol.GetUpperAndLowerLimit(upperTol, lowerTol);
    Debug.Print("        Upper / Lower   " + upperTol + " / " + lowerTol);
    Debug.Print("        Block tolerance precision = " + myDimTol.BlockToleranceDecimalPlaces);
    Debug.Print("        Limits and fits code = " + myDimTol.LimitsAndFitsCode);
}
Debug.Print(" ");
}
}
}
}
}
}
}
public SldWorks swApp;
}
}

```